

OS project2 wiki

컴퓨터소프트웨어학부

2021018068 정환

1. Design

명세에 맞춰 한 프로세스의 스레드끼리 address space를 공유하기 위해 pagetable을 공유하게 했다. context와 trapframe을 스레드마다 따로 갖게 하고, 명세의 특징에 따라 파일 디스크립터 테이블도 따로 갖게 했다.

스레드를 나타내는 자료구조로는 기존에 있는 struct proc을 사용했다. 대신 필드에 스레드를 구현하는데 필요한 것들을 넣었다.

2. Implementation

a) proc.h

```
84 // Per-process state
85 struct proc {
86     struct spinlock lock;
87
88     // p->lock must be held when using these:
89     enum procstate state; // Process state
90     void *chan;           // If non-zero, sleeping on chan
91     int killed;           // If non-zero, have been killed
92     int xstate;           // Exit status to be returned to parent's wait
93     int pid;              // Process ID
94
95     // wait_lock must be held when using this:
96     struct proc *parent; // Parent process
97
98     // for thread
99     struct proc *mainthread; // main thread
100     int tid; // thread id in a process
101     int tcnt; // to create tid
102     void *raw_stack; // not aligned stack addr
103
104     // these are private to the process, so p->lock need not be held.
105     uint64 kstack; // Virtual address of kernel stack
106     uint64 sz; // Size of process memory (bytes)
107     uint64 *sz_ptr; // pointer of Size of main thread memory
108     pagetable_t pagetable; // User page table
109     uint64 trapframe_va; // virtual address of the trapframe
110     struct trapframe *trapframe; // data page for trampoline.S
111     struct context context; // swtch() here to run process
112     struct file *ofile[NOFILE]; // Open files
113     struct inode *cwd; // Current directory
114     char name[16]; // Process name (debugging)
115 };
```

proc 구조체에 mainthread, tid, tcnt, raw_stack, sz_ptr을 추가했다. 스레드가 어떤 프로세스에 속하는지를 mainthread로 구분한다. trapframe을 pagetable에 매핑해주기 위해 tid를 스레드마다 설정하고, tcnt를 이용해 tcnt를 정해준다. tcnt는 메인 스레드에서만 변경시키며 사용한다. raw_stack은 thread_create에 유저에서 메모리 할당한 stack 주소를 저장해두기 위해 사용한다. page align하기 전의 시작 주소를

raw_stack에 저장해둔다. sz는 페이지 테이블로 매핑된 유저 주소 공간의 크기이므로 메인스레드의 sz를 공유하기 위해 메인스레드의 sz의 주소를 저장하는 sz_ptr을 만들었다.

b) defs.h

```
109 int clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack);
110 int join(void **stack);
111 void terminatethreads(struct proc*);
```

clone, join, terminatethreads함수를 defs.h에 선언했다. terminatethreads함수는 exec()을 호출할 때 호출한 스레드를 제외한 다른 스레드를 모두 종료시키기 위해 만들었다.

c) proc.c

- allocproc(void)

```
146 // Set up new context to start executing at forkret,
147 // which returns to user space.
148 memset(&p->context, 0, sizeof(p->context));
149 p->context.ra = (uint64)forkret;
150 p->context.sp = p->kstack + PGSIZE;
151
152 // set mainthread
153 p->mainthread = p;
154 p->tcnt = 1;
155 p->tid = 1;
156
157 return p;
158 }
```

프로세스를 만들 때 mainthread를 본인, tcnt, tid를 1로 설정했다.

- freeproc(struct proc *p)

```
174 p->sz_ptr = 0;
175 p->pid = 0;
176 p->parent = 0;
177 p->name[0] = 0;
178 p->chan = 0;
179 p->killed = 0;
180 p->xstate = 0;
181 p->state = UNUSED;
182 p->mainthread = 0;
183 p->tid = 0;
184 p->tcnt = 0;
```

프로세스의 자원을 회수할 때 mainthread, tid, tcnt, sz_ptr을 0으로 초기화하는 걸 추가했다. sz_ptr은 userinit()과 fork()에서 설정한다.

- userinit(void)

```
255     uvmfirst(p->pagetable, initcode, sizeof(initcode));
256     p->sz = PGSIZE;
257     p->sz_ptr = &(p->sz);
```

initprocess를 만들 때 p->sz_ptr를 p->sz의 주솟값으로 설정했다. 본인이 메인스레드이기 때문이다.

- fork(void)

```
306     if(uvmcopy(p->pagetable, np->pagetable, *(p->sz_ptr)) < 0){
307         freeproc(np);
308         release(&np->lock);
309         return -1;
310     }
311     np->sz = *(p->sz_ptr);
312     np->sz_ptr = &(np->sz);
```

스레드에서 fork를 수행할 수도 있으므로 p->sz대신 *(p->sz_ptr)로 pagetable 복사를 실시한다. 새로운 프로세스의 sz와 sz_ptr도 본인이 메인 스레드가 되는 것이므로 위와 같이 설정한다.

- growproc(int n)

```
273     int
274     growproc(int n)
275     {
276         uint64 sz;
277         struct proc *p = myproc();
278
279         sz = *(p->sz_ptr);
280         if(n > 0){
281             if((sz = uvmmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
282                 return -1;
283             }
284         } else if(n < 0){
285             sz = uvmdalloc(p->pagetable, sz, sz + n);
286         }
287         *(p->sz_ptr) = sz;
288         return 0;
289     }
```

유저 가상 공간을 늘릴때도 sz와 sz_ptr을 적절히 사용하게 했다.

- clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack)

```
838 int
839 clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack){
840     int pid;
841     struct proc *nthread;
842
843     if((nthread = allocthread()) == 0){
844         return -1;
845     }
846
847     nthread->trapframe->epc = (uint64)fcn;
848     nthread->trapframe->a0 = (uint64)arg1;
849     nthread->trapframe->a1 = (uint64)arg2;
850
851     void *aligned_stack = (void *)(((uint64)stack + PGSIZE - 1) & ~(PGSIZE - 1));
852     nthread->trapframe->sp = (uint64)aligned_stack + PGSIZE;
853
854     nthread->raw_stack = stack;
855
856     pid = nthread->pid;
857     nthread->state = RUNNABLE;
858
859     release(&nthread->lock);
860
861     return pid;
862 }
```

allocthread()를 호출하여 새로운 스레드를 할당한다. 그리고 받은 파라미터들을 새로운 스레드에 적절히 넣어준다. stack은 2*PGSIZE로 유저 공간에서 메모리를 할당하여 온다. 이때 page aligned된 stack 주소값을 만들기 위해 aligned_stack을 위와 같이 설정한다. 그리고 stack은 위에서 아래로 자라기 때문에 초기값으로 aligned_stack에 사용자 스택의 크기인 PGSIZE를 더해 넣어 가장 위쪽을 가르키게 한다. 그 후 state를 RUNNABLE로 바꾼 뒤 pid를 return 한다.

- allocthread(void)

```
746 static struct proc*
747 allocthread(void)
748 {
749     int i;
750     struct proc *nthread;
751     struct proc *thread = myproc();
752     struct proc *mainthread;
753
754     for(nthread = proc; nthread < &proc[NPROC]; nthread++) {
755         acquire(&nthread->lock);
756         if(nthread->state == UNUSED) {
757             goto found;
758         } else {
759             release(&nthread->lock);
760         }
761     }
762     return 0;
763
764 found:
```

proc 배열을 돌며 UNUSED인 프로세스 공간이 있다면 새로운 스레드를 할당한다.

```

764 found:
765     nthread->pid = allocpid();
766     nthread->state = USED;
767     nthread->parent = 0;
768
769     acquire(&thread->lock);
770     // set main thread
771     mainthread = thread->mainthread;
772     nthread->mainthread = mainthread;
773     // assign main thread's pagetable
774     nthread->pagetable = thread->pagetable;
775     release(&thread->lock);
776
777     acquire(&mainthread->lock);
778     nthread->sz_ptr = mainthread->sz_ptr;
779     nthread->sz = mainthread->sz;
780     nthread->tid = ++mainthread->tcnt; // set tid
781     release(&mainthread->lock);
782
783
784     // Allocate a trapframe page.
785     if((nthread->trapframe = (struct trapframe *)kalloc()) == 0){
786         freethread(nthread);
787         release(&nthread->lock);
788         return 0;
789     }
790
791     // map new thread's trapframe to pagetable
792     if(mappages(nthread->pagetable, nthread->trapframe_va = TRAMPOLINE - (nthread->tid * PGSIZE),
793         PGSIZE, (uint64)(nthread->trapframe), PTE_R | PTE_W) < 0){
794         freethread(nthread);
795         release(&nthread->lock);
796         return 0;
797     }

```

스레드의 pid를 할당하고 USED로 바꾼 뒤 parent를 0으로 설정한다. 메인 스레드가 아닌 스레드만 allocthread로 생성되기 때문에 부모가 누군지 설정하지 않았다. main thread를 현재 실행중인 프로세스(스레드)의 main thread로 설정하고 pagetable을 공유하게 만들었다. 또한 sz와 sz_ptr도 mainthread->sz를 이용하게 만들었다. tid는 mainthread의 tcnt를 이용해 설정해주었다.

스레드의 trapframe은 따로 가져야하기 때문에 kalloc을 이용해 메모리 할당해주었다. 스레드 간에 공유하는 pagetable에 trapframe을 매핑하는 건 trapframe의 가상주소를 TRAMPOLINE - (nthread->tid * PGSIZE)로 설정해서 스레드 간 trapframe이 겹치지 않게 했다.

```

799 // Set up new context to start executing at forkret,
800 // which returns to user space.
801 memset(&nthread->context, 0, sizeof(nthread->context));
802 nthread->context.ra = (uint64)forkret;
803 nthread->context.sp = nthread->kstack + PGSIZE;
804
805 // increment reference counts on open file descriptors.
806 for(i = 0; i < NOFILE; i++)
807     if(thread->ofile[i])
808         nthread->ofile[i] = filedup(thread->ofile[i]);
809 nthread->cwd = idup(thread->cwd);
810
811 safestrcpy(nthread->name, thread->name, sizeof(thread->name));
812
813 return nthread;
814 }

```

그 후 context를 초기화 하고, 스레드의 시작 위치를 설정해주었다. 파일 디스크립터 테이블을 스레드끼리 다르게 가지기 때문에 새로 만든 스레드의 파일 디스크립터 테이블의 항목이 현재 스레드의 파일 객체를 가르키게 만들었다. 그 후 이름을 설정 후 스레드를 return 한다.

- join(void **stack)

```

864 int
865 join(void **stack){
866     struct proc *cthread;
867     int havekids, pid;
868     struct proc *thread = myproc();
869
870     acquire(&wait_lock);
871
872     for(;;){
873         havekids = 0;
874         for(cthread = proc; cthread < &proc[NPROC]; cthread++){
875             if(cthread->mainthread == thread && cthread != thread){
876                 acquire(&cthread->lock);
877
878                 havekids = 1;
879                 if(cthread->state == ZOMBIE){
880                     // Found one.
881                     pid = cthread->pid;
882
883                     copyout(cthread->pagetable, (uint64)stack, (char *)&cthread->raw_stack, sizeof(cthread->raw_stack));
884
885                     freethread(cthread);
886                     release(&cthread->lock);
887                     release(&wait_lock);
888                     return pid;
889                 }
890                 release(&cthread->lock);
891             }
892         }
893
894         // No point waiting if we don't have any children.
895         if(!havekids || killed(thread)){
896             release(&wait_lock);
897             return -1;
898         }
899
900         // Wait for a child to exit.
901         sleep(thread, &wait_lock); //DOC: wait-sleep
902     }
903 }

```

join은 메인 스레드가 호출해야 의미있는 행동을 하게 만들었다. 자신을

메인스레드로 가지는 자식 스레드들을 찾고, ZOMBIE라면 자원을 회수한다. 만약 자식이 있는데 ZOMBIE인 스레드가 없다면 sleep을 호출해 기다린다. 자식이 없다면 -1을 return한다. 파라미터로 받은 stack에는 해제해야할 메모리 주소인 스레드의 raw_stack을 넘겼다. 자원 회수는 freethread()에서 실시한다.

-freethread(struct proc *thread)

```
816 void
817 freethread(struct proc *thread)
818 {
819     uvmunmap(thread->pagetable, thread->trapframe_va, 1, 0);
820     if(thread->trapframe)
821         kfree((void*)thread->trapframe);
822     thread->trapframe = 0;
823     thread->trapframe_va = 0;
824     thread->pagetable = 0;
825     thread->sz = 0;
826     thread->sz_ptr = 0;
827     thread->pid = 0;
828     thread->parent = 0;
829     thread->name[0] = 0;
830     thread->chan = 0;
831     thread->killed = 0;
832     thread->xstate = 0;
833     thread->state = UNUSED;
834     thread->mainthread = 0;
835     thread->tid = 0;
836 }
```

pagetable을 스레드끼리 공유하므로 pagetable을 free해주면 안된다. trapframe은 따로 가지기 때문에 pagetable에 매핑한 trapframe을 unmap해준 뒤 trapframe을 메모리 해제 해준다. 그 후 다른 특성을 초기화해준다.

- exit(int status)

```
361 void
362 exit(int status)
363 {
364     struct proc *p = myproc();
365     struct proc *mainthread;
366
367     acquire(&p->lock);
368     mainthread = p->mainthread;
369     release(&p->lock);
370
371     if(p == initproc)
372         panic("init exiting");
373
374     // Close all open files.
375     for(int fd = 0; fd < NOFILE; fd++){
376         if(p->ofile[fd]){
377             struct file *f = p->ofile[fd];
378             fileclose(f);
379             p->ofile[fd] = 0;
380         }
381     }
382
383     begin_op();
384     iput(p->cwd);
385     end_op();
386     p->cwd = 0;
387
388     acquire(&wait_lock);
389
```

현재 스레드의 메인 스레드를 찾는 코드를 추가했다. 그 후 파일 디스크립터 테이블이 가르키는 파일을 닫고, 0으로 초기화했다.


```

390     if(mainthread == p){
391         struct proc *cthread;
392
393         for(cthread = proc; cthread < &proc[NPROC]; cthread++){
394             acquire(&cthread->lock);
395             if(cthread->mainthread == p && cthread != p){
396                 cthread->killed = 1;
397                 if(cthread->state == SLEEPING){
398                     cthread->state = RUNNABLE;
399                 }
400             }
401             release(&cthread->lock);
402         }
403
404         // Parent might be sleeping in wait().
405         wakeup(p->parent);
406     } else {
407         wakeup(mainthread);
408     }
409
410     // Give any children to init.
411     reparent(p);
412
413     release(&wait_lock);
414
415     if(mainthread == p){
416         while(join(0) != -1);
417     }
418
419     acquire(&p->lock);
420     p->xstate = status;
421     p->state = ZOMBIE;
422
423     // Jump into the scheduler, never to return.
424     sched();
425     panic("zombie exit");
426 }

```

만약 mainthread가 본인이라면 프로세스 내 다른 스레드들도 종료시켜야 한다. 본인을 메인스레드로 가지는 스레드들을 찾아 killed를 1로 설정하고 자고 있다면 RUNNABLE로 바꾼다. 그 후 현재 스레드의 부모 스레드를 깨운다. killed = 1인 스레드는 커널로 진입하면 killed = 1인걸 확인한 후 exit()을 호출한다. 메인스레드의 자원이 회수되기 전에 다른 스레드들의 unmap과 자원 회수가 이루어져야하기 때문에 while(join(0) != -1)을 사용했다.

메인스레드가 아닌 스레드가 exit()을 호출한거라면 스레드의 메인스레드를 깨운다.

그 이후에는 현재 스레드의 자식 프로세스들을 reparent해주고, 현재 스레드를 ZOMBIE로 만든다.

- kill(int pid)

```
637 int
638 kill(int pid)
639 {
640     struct proc *p;
641     struct proc *thread;
642     struct proc *mainthread;
643
644     for(p = proc; p < &proc[NPROC]; p++){
645         acquire(&p->lock);
646         if(p->pid == pid){
647             mainthread = p->mainthread;
648             release(&p->lock);
649
650             for(thread = proc; thread < &proc[NPROC]; thread++){
651                 acquire(&thread->lock);
652                 if(thread->mainthread == mainthread){
653                     thread->killed = 1;
654                     if(thread->state == SLEEPING){
655                         // Wake process from sleep().
656                         thread->state = RUNNABLE;
657                     }
658                 }
659                 release(&thread->lock);
660             }
661
662             return 0;
663         }
664         release(&p->lock);
665     }
666     return -1;
667 }
```

한 스레드가 kill되면 모든 스레드를 종료시키는 작업을 구현했다. kill할 스레드를 찾고 그 스레드가 속하는 프로세스의 모든 스레드의 killed를 1로 설정한다.

- terminatethreads(struct proc *mainthread)

```
907 void
908 terminatethreads(struct proc *mainthread){
909     struct proc *thread;
910     struct proc *p = myproc();
911
912     for(thread = proc; thread < &proc[NPROC]; thread++) {
913         acquire(&thread->lock);
914         if(thread->mainthread == mainthread && thread != p) {
915             thread->mainthread = p;
916             thread->killed = 1;
917             if(thread->state == SLEEPING){
918                 thread->state = RUNNABLE;
919             }
920         }
921         release(&thread->lock);
922     }
923
924     while(join(0) != -1);
925 }
```

파라미터로 받은 메인스레드를 메인스레드로 하는 스레드를 찾아 mainthread를 현재 실행중인 스레드로 바꾸고 killed=1로 설정한다. 그 후 자원회수까지 끝나는 걸 기다린다. mainthread를 현재 스레드로 바꾸고 난

뒤 이 함수를 호출하기 때문에 join()이 정상적으로 작동한다. 이 함수는 exec()안에서만 쓰인다. proc배열을 exec.c 안에서 찾지 못해 proc.c 안에 이 함수를 만들었다.

d) exec.c

```
22 int
23 exec(char *path, char **argv)
24 {
25     char *s, *last;
26     int i, off;
27     uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
28     struct elfhdr elf;
29     struct inode *ip;
30     struct proghdr ph;
31     pagetable_t pagetable = 0, oldpagetable;
32     struct proc *p = myproc();
33     struct proc *oldmainthread;
34
35     acquire(&p->lock);
36     oldmainthread = p->mainthread;
37     p->mainthread = p;
38     p->sz = *(p->sz_ptr);
39     p->sz_ptr = &(p->sz);
40
41     if(oldmainthread != p){
42         acquire(&oldmainthread->lock);
43         p->parent = oldmainthread->parent;
44         release(&oldmainthread->lock);
45     }
46     release(&p->lock);
47
48     if(oldmainthread != p){
49         terminatethreads(oldmainthread);
50     }
51
52     acquire(&p->lock);
53     uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
54     mappages(p->pagetable, p->trapframe_va = TRAMPOLINE - PGSIZE,
55             PGSIZE, (uint64)(p->trapframe), PTE_R | PTE_W);
56     release(&p->lock);
57 }
```

exec()을 호출한 스레드를 메인스레드로 만들고, 다른 스레드를 종료시키는 과정을 수행한다. 바꾼 메인스레드의 부모를 기존 메인스레드의 부모로 설정하는 방법을 사용했다. 또한 바꾼 메인스레드의 trapframe_va가 TRAMPOLINE값이 아니기 때문에 코드 뒤 쪽에서 페이지테이블을 해제할 때 에러가 발생할 수 있으므로 trapframe_va를 TRAMPOLINE - PGSIZE로 사용해 다시 매핑해주었다.

```

148 // Commit to the user image.
149 oldpagetable = p->pagetable;
150 p->pagetable = pagetable;
151 p->sz = sz;
152 p->sz_ptr = &(p->sz);
153 p->trapframe->epc = elf.entry; // initial program counter = main
154 p->trapframe->sp = sp; // initial stack pointer
155 p->mainthread = p;
156 p->tcnt = 1;
157 p->tid = 1;
158
159 proc_freepagetable(oldpagetable, oldsz);
160
161 return argc; // this ends up in a0, the first argument to main(argc, argv)

```

뒤쪽에서 프로세스를 초기화 할 때 스레드를 위한 값들을 설정해주었다.

- sysproc.c

```

96 uint64
97 sys_clone(void)
98 {
99     uint64 fcn, arg1, arg2, stack;
100
101     argaddr(0, &fcn);
102     argaddr(1, &arg1);
103     argaddr(2, &arg2);
104     argaddr(3, &stack);
105
106     return clone((void (*)(void*, void*))fcn, (void*)arg1, (void*)arg2, (void*)stack);
107 }
108
109 uint64
110 sys_join(void)
111 {
112     uint64 stack;
113
114     argaddr(0, &stack);
115
116     return join((void **)stack);
117 }

```

clone과 join의 시스템 콜 함수를 이렇게 만들었다. 나머지 시스템 콜 세부 구현은 생략하겠다.

- user/thread.h

```

1 #define PGSIZE 4096
2
3 int thread_create(void (*start_routine)(void*, void*), void *arg1, void *arg2);
4 int thread_join();

```

thread.h에 PGSIZE를 define하고, thread_create, thread_join 함수를 선언했다.

- user/thread.c

```
1  #include "kernel/types.h"
2  #include "user/thread.h"
3  #include "user/user.h"
4
5  int thread_create(void (*start_routine)(void*, void*), void *arg1, void *arg2)
6  {
7      void *raw_stack = malloc(PGSIZE * 2);
8
9      int tid = clone(start_routine, arg1, arg2, raw_stack);
10
11     return tid;
12 }
13
14 int thread_join() {
15     void *stack;
16     int tid = join(&stack);
17
18     if(tid >= 0){
19         free(stack);
20     }
21
22     return tid;
23 }
```

thread_create에서 PGSIZE * 2만큼 메모리를 할당해준다. page align을 해주기 위해 여윳값을 두고 할당한다. 그 후 raw_stack과 파라미터들로 clone을 호출한다. clone 내에서 raw_stack은 proc 구조체에 저장되고 page aligned stack을 연산하여 sp로 사용한다.

thread_join은 thread_create에서 할당했던 raw_stack을 받아와 메모리 해제한다.

3. Results

```
[TEST#1]
Thread 0 start
Thread 1 start
Thread 1 end
Thread 2 start
Thread 2 end
Thread 3 start
Thread 3 end
Thread 4 start
Thread 4 end
Thread 0 end
TEST#1 Passed

[TEST#2]
Thread 0 start, iter=0
Thread 0 end
Thread 1 start, iter=1000
Thread 1 end
Thread 2 start, iter=2000
Thread 2 end
Thread 3 start, iter=3000
Thread 3 end
Thread 4 start, iter=4000
Thread 4 end
TEST#2 Passed

[TEST#3]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#3 Passed

[TEST#4]
Thread 0 sbrk: old break = 0x0000000000015000
Thread 0 sbrk: increased break by 14000
new break = 0x0000000000029010
Thread 1 size = 0x0000000000029010
Thread 2 size = 0x0000000000029010
Thread 3 size = 0x0000000000029010
Thread 4 size = 0x0000000000029010
Thread 0 sbrk: free memory
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#4 Passed

[TEST#5]
Thread 0 start, pid 29
Thread 1 start, pid 29
Thread 2 start, pid 29
Thread 3 start, pid 29
Thread 4 start, pid 29
TEST#5 Passed

[TEST#6]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Thread exec test 0
TEST#6 Passed

All tests passed. Great job!!
$
```

테스트 코드에 대해 정상적으로 작동하는 것을 확인했다.

4. Troubleshooting

스레드들의 자원을 회수하는 순서를 맞춰주는 게 어려웠다. 커널에서 강제로 다른 스레드를 종료시켜야 할 때 `join(0)`을 이용해서 스레드들의 자원 회수가 끝나는걸 기다렸는데 유저 공간에서 메모리를 할당한 `stack`을 커널이 메모리 해제를 할 수가 없다. 사용자가 잘못 사용하지 않는다는 전제하에 진행해야 하는 것으로 가정을 하고 구현했다.